

Java Database Connectivity (JDBC)

Crud Operation with JDBC :-

- JDBC stands for Java Database Connectivity.
- It allows Java programs to connect with relational databases.
- Databases like PostgreSQL, MySQL, or Oracle can be used.
- It provides interfaces and classes to execute SQL queries.

What are CRUD Operation ?

CRUD stands for:

1. **C (Create)** -> Insert new records into the database.
2. **R (Read)** -> Retrieve records from the database.
3. **U (Update)** -> Modify existing records.
4. **D (Delete)** -> Remove records from the database.

Steps to Perform CRUD Operations in JDBC :-

Step	Description
1. Load JDBC Driver	The driver class is loaded using <code>Class.forName("org.postgresql.Driver");</code> . It registers the driver with the <code>DriverManager</code> .
2. Establish Connection	A connection to the database is created using <code>DriverManager.getConnection(URL, username, password);</code> .
3. Create Statement / PreparedStatement	SQL queries are sent to the database using <code>Statement</code> or <code>PreparedStatement</code> objects.
4. Execute Query	The SQL command (<code>INSERT</code> , <code>SELECT</code> , <code>UPDATE</code> , <code>DELETE</code>) is executed using methods like <code>executeUpdate()</code> or <code>executeQuery()</code> .
5. Process Results	Results returned by <code>SELECT</code> queries are accessed using the <code>ResultSet</code> object.
6. Close Connection	All database resources are closed using <code>close()</code> to prevent memory leaks.

1. Create Operation (INSERT) :-

- Used to insert new records into a table.
- Executed using executeUpdate() method.

Example :

```
package JDBCPROGRAMS;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class insert_operation {

    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        //1.load class of jdbc driver
        Class.forName("org.postgresql.Driver");
        System.out.println("1.JDBC Driver Loaded!");

        //2.create connection portname,username & password
        //rdbmsdriver://localhost:portnumber/databasename
        String URL="jdbc:postgresql://localhost:5433/java8sept";
        String username="postgres";
        String password="4518";
        Connection con=DriverManager.getConnection(URL,username,password);
        System.out.println("2.Connection is Created");

        //3.create a statement
        String insert="insert into student values(1,'Sagar'),(2,'Rahul')";

        Statement st=con.createStatement();
        System.out.println("3.Statement is Created");

        //4.execute the query
        st.executeUpdate(insert);
        System.out.println("4.Query Executed");

        //5.close the query
        con.close();
        System.out.println("5.Close the Connection");
    }
}
```

Output :-

```
1.JDBC Driver Loaded!
2.Connection is Created
3.Statement is Created
4.Query Executed
5.Close the Connection
```

2. Update Operation (UPDATE) :-

- Used to modify existing records.
- Executed using executeUpdate() method.

Example:

```
package JDBCPROGRAMS;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class update_operation {

    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        //1.load class of jdbc driver
        Class.forName("org.postgresql.Driver");
        System.out.println("1.JDBC Driver Loaded!");

        //2.create connection portname,username & password
        //rdbmsdriver://localhost:portnumber/databasename
        String URL="jdbc:postgresql://localhost:5433/java8sept";
        String username="postgres";
        String password="4518";
        Connection con=DriverManager.getConnection(URL,username,password);
        System.out.println("2.Connection is Created");

        //3.create a statement
        String update="update student set name='Ganesh' where id=1";

        Statement st=con.createStatement();
        System.out.println("3.Statement is Created");

        //4.execute the query
        st.executeUpdate(update);
        System.out.println("4.Query Executed");

        //5.close the query
        con.close();
        System.out.println("5.Close the Connection");
    }
}
```

Output :-

```
1.JDBC Driver Loaded!
2.Connection is Created
3.Statement is Created
4.Query Executed
5.Close the Connection
```

3. Delete Operation (DELETE) :-

- Used to remove records from a table.
- Executed using executeUpdate() method.

Example :

```
package JDBCPROGRAMS;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class delete_operation {

    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        //1.load class of jdbc driver
        Class.forName("org.postgresql.Driver");
        System.out.println("1.JDBC Driver Loaded!");

        //2.create connection portname,username & password
        //rdbmsdriver://localhost:portnumber/databasename
        String URL="jdbc:postgresql://localhost:5433/java8sept";
        String username="postgres";
        String password="4518";
        Connection con=DriverManager.getConnection(URL,username,password);
        System.out.println("2.Connection is Created");

        //3.create a statement
        String delete="delete from student where id=1";

        Statement st=con.createStatement();
        System.out.println("3.Statement is Created");

        //4.execute the query
        st.executeUpdate(delete);
        System.out.println("Query Executed");

        //5.close the query
        con.close();
        System.out.println("5.Close the Connection");

    }

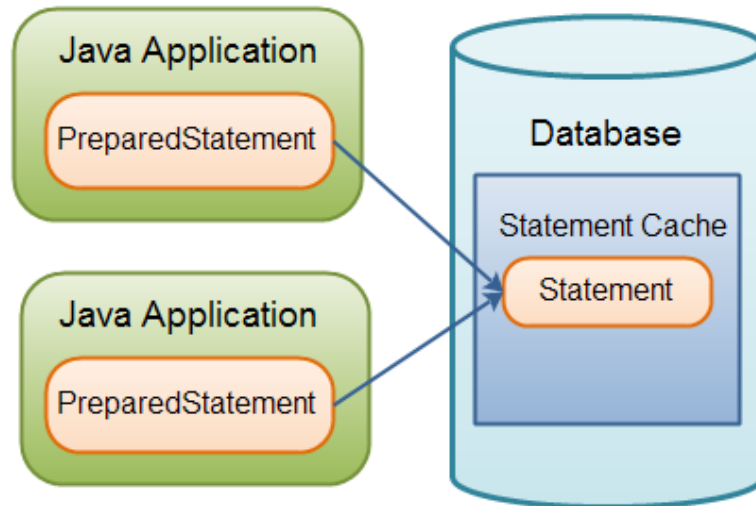
}
```

Output :-

```
1.JDBC Driver Loaded!
2.Connection is Created
3.Statement is Created
4.Query Executed
5.Close the Connection
```

Prepared Statement :-

- PreparedStatement in Java is a precompiled SQL statement.
- It is a subinterface of the Statement interface.
- It offers better performance and cleaner code.
- It protects against SQL injection attacks.



Why Use Prepared Statement ?

1. **Performance Improvement:** Queries are compiled once and reused.
2. **Reusability:** The same query can be executed with different parameter values.
3. **Security:** Prevents SQL injection by safely binding user input.
4. **Code Readability:** Avoids messy string concatenations in SQL queries.

Steps to use Prepared Statement :-

1. Load the JDBC driver.
2. Establish a connection using `DriverManager.getConnection()`.
3. Create a `PreparedStatement` object with an SQL query containing placeholders `(?)`.
4. Set parameter values using methods like `setInt()`, `setString()`, etc.
5. Execute the query using `executeUpdate()` or `executeQuery()`.
6. Process the result if applicable.
7. Close the `PreparedStatement` and database connection.

Common Methods in Prepared Statement :-

Method	Use
setInt()	Set integer value
setString()	Set string value
setFloat()	Set float value
setDouble()	Set double value
executeQuery()	Run SELECT query
executeUpdate()	Run INSERT/UPDATE/DELETE
execute()	Run any SQL query
close()	Close statement

Example :-

```
package JDBCPROGRAMS;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;

public class prepare_statement {

    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        //1.load class of jdbc driver
        Class.forName("org.postgresql.Driver");
        System.out.println("1.JDBC Driver Loaded!");

        //2.create connection portname,username & password
        //rdbmsdriver://localhost:portnumber/databasename
        String URL="jdbc:postgresql://localhost:5433/java8sept";
        String username="postgres";
        String password="4518";
        Connection con=DriverManager.getConnection(URL,username,password);
        System.out.println("2.Connection is Created");

        //3.create a ststatement using prepare statement
        PreparedStatement ps=con.prepareStatement("insert into student values(?,?)");
        Scanner sc=new Scanner(System.in);

        System.out.println("Enter Id: ");
        int id=sc.nextInt();

        System.out.println("Enter Name:");
        String name=sc.next();

        ps.setInt(1,id);
        ps.setString(2,name);

        System.out.println("3.Statement is Created");

        //4.execute the query
        ps.executeUpdate();
        System.out.println("4.Query is Created");

        //5.close the connection
        con.close();
        System.out.println("5.Close the Connection");

    }
}
```

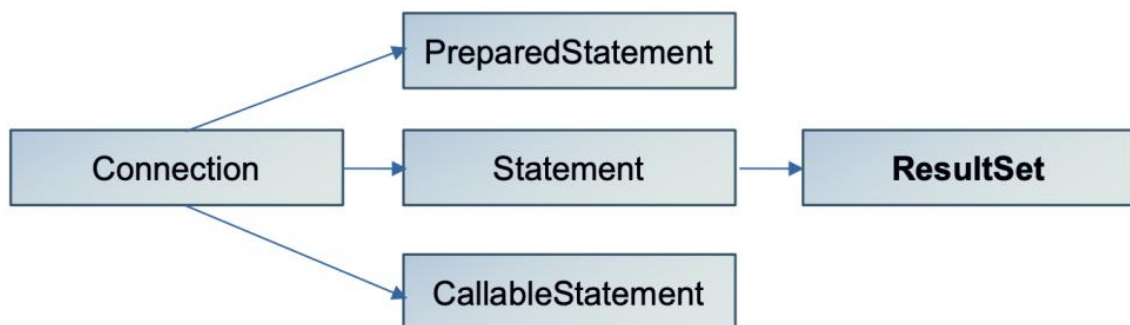
Output :-

```
1.JDBC Driver Loaded!
2.Connection is Created
3.Statement is Created
4.Query Executed
5.Close the Connection
```

ResultSet :-

What is ResultSet ?

- ResultSet is an interface in the java.sql package.
- It represents a set of rows returned by executing a SQL SELECT query.
- It acts as a cursor that allows Java programs to iterate over query results.
- ResultSet is created when the executeQuery() method of Statement or PreparedStatement is called.



Creation of ResultSet :-

- A ResultSet object is usually created as follows:

```
Statement st = con.createStatement();
```

```
ResultSet rs = st.executeQuery("SELECT * FROM student");
```

Implementation of ResultSet :-

1. Load the JDBC driver using Class.forName().
2. Establish a database connection using DriverManager.getConnection().
3. Create a Statement object.
4. Execute a SELECT query using executeQuery().
5. Store results in a ResultSet object.
6. Use next() and getXXX() methods to read data.
7. Close ResultSet, Statement, and Connection objects.

Example :-

```
package JDBCPROGRAMS;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class resultset {

    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        //1.load class of jdbc driver
        Class.forName("org.postgresql.Driver");
        System.out.println("1.JDBC Driver Loaded!");

        //2.create connection portname,username & password
        //rdbmsdriver://localhost:portnumber/databasename
        String URL="jdbc:postgresql://localhost:5433/java8sept";
        String username="postgres";
        String password="4518";
        Connection con=DriverManager.getConnection(URL,username,password);
        System.out.println("2.Connection is Created");

        //3.create a statement
        Statement st=con.createStatement();
        String fetch="select * from student";
        System.out.println("3.Statement is Created");

        //4.execute query
        ResultSet rs=st.executeQuery(fetch);
        System.out.println("4.Execute Query");

        //5.retrive
        while(rs.next())
        {
            System.out.println(rs.getInt(1));
            System.out.println(rs.getString(2));
        }

        //6.close the connection
        con.close();
        System.out.println("5.Close the Connection");
    }
}
```

Output :-

```
1.JDBC Driver Loaded!
2.Connection is Created
3.Statement is Created
4.Execute Query
2
Rahul
```

301

PK

302

SK

1

Sagar

2

Rahul

1

Sagar

2

Rahul

5.Close the Connection

	id integer 	name character varying (100) 
1	2	Rahul
2	301	PK
3	302	SK
4	1	Sagar
5	2	Rahul
6	1	Sagar
7	2	Rahul